

Table 4: Monitoring and logging in microservices architecture

Component	Description	Common tools	Best practices
Centralised logging	Aggregates logs from all microservices into a single platform for easier search and analysis.	ELK Stack (Elasticsearch, Logstash, Kibana), Fluentd, Graylog	Use structured logging and consistent log formats across services.
Metrics collection	Collects quantitative data on system performance (CPU, memory, latency, error rates).	Prometheus, Grafana	Expose metrics endpoints and monitor critical service KPIs regularly.
Distributed tracing	Tracks requests as they flow through multiple microservices, identifying bottlenecks and failures.	Jaeger, Zipkin	Use correlation IDs to link traces across services for full visibility.
Alerting	Sends notifications based on metric thresholds or anomaly detection to alert teams proactively.	Prometheus Alertmanager, PagerDuty, Opsgenie	Define clear, actionable alerts to avoid alert fatigue and ensure timely incident response.
Health checks	Regular checks on service status to ensure availability and readiness for handling requests.	Kubernetes readiness/liveness probes, custom scripts	Automate health checks to trigger restarts or traffic routing decisions based on service health.
Secure data handling	Ensure logs and metrics are stored securely with appropriate access controls and retention policies.	Encryption, role-based access control (RBAC)	Implement data privacy compliance, secure storage, and limit access to sensitive information.

Selecting the correct protocol: gRPC provides better performance and supports bi-directional streaming, but it requires more setup than REST, which is widely used and language-neutral.

Service discovery

Static service endpoints are impractical in dynamic environments where services scale up or down or where IP addresses change frequently. This is resolved by service discovery, which makes it possible for services to dynamically find one another.

Client-side discovery: To find the location of the service instance, the client sends a query to a service registry (such as Consul or Eureka).

Server-side discovery: By querying the registry and rerouting requests appropriately, a load balancer or API gateway abstracts client discovery.

In containerised environments, this process is made simpler by Kubernetes' integrated DNS-based service discovery feature.

Here are some additional considerations:

- To maximise resource utilisation and avoid overload, load balancing divides incoming requests equally among service instances.
- **Health checks:** Make sure traffic is only directed to healthy instances by routinely checking service responsiveness and availability.
- **Timeouts and retries:** Putting timeouts and retry policies in place increases system robustness overall and stops

cascading failures.

Building a scalable microservices architecture with Docker is a tactical solution to the diversified needs of modern app development. The containerisation offered by Docker provides a uniform, lightweight, and portable environment that makes deploying easy and sure to operate as expected, no matter the stage of deployment considered — be it development or production.

Moreover, an integrated monitoring and logging setup keeps an eye on the health and performance of distributed services, allowing early detection of issues and quicker remediation. These practices, in view of automated scaling and self-healing, are crucial factors enforcing resilient techniques and competing uptimes.

This architecture, if followed, promotes an independent development and deployment cycle that lets teams test new ideas fast without jeopardising the stability of the whole system. The complexity it entails can be harnessed by using Docker's ecosystem to employ some best practices in the areas of orchestration, communication, and observability. **END** 

By: Rupam Bhattacharjee

The author is a data and analytics professional with 12 years of experience in the industry. He specialises in machine learning, artificial intelligence, and data-driven solutions, and is passionate about leveraging emerging technologies like generative AI to solve complex problems and drive innovation. His expertise spans Python programming, data management, and automation.